

Теми за Държавен Изпит

Д. Иванов, КН

11 юли 2025 г.

Функционално програмиране

19. Функционално програмиране. Обща характеристика на функционалния стил на програмиране. Дефиниране и използване на функции. Модели на оценяване. Функции от по-висок ред.

1. Характеристика на функционалния стил. Основни компоненти. Прimitives изрази. Дефиниране на функции. Оценяване.

Във функционалния стил на програмиране изчислението се свежда до дефиниране на функции и оценка на изрази – обръщания към функции. При него не се използват оператори за присвояване, за цикли или предаване на управлението. Позволено е единствено извикване на функции, които пресмятат и връщат стойности, базирани единствено на подадените аргументи (фактически параметри). Функциите (т.н. "чисти" или строги) нямат никакъв страничен ефект - обектите в строгите функционални езици не се изменят и на практика нямат състояние (immutable). Този стил на програмиране се основава на ламбда смятането. Това е математическата теория, разработена за да подпомогне изследването на работата с функции и рекурсивната обработка.

Предимства на функционалното програмиране:

- може да се извършва лесна проверка и поправка на съответните програми поради липсата на странични ефекти
- подходящ при проектирането на езици за програмиране, предназначени за многопроцесорни компютри, в които много от пресмятанията се извършват паралелно (поради липсата на странични ефекти не се налага програмистът да се грижи за евентуални грешки, причинени от промени на стойности, извършени в неправилен ред)
- свойства на функционалните програми могат да бъдат доказвани точно (с математически средства)

Недостатъци на функционалното програмиране:

- строгата функционалност понякога изисква многократно пресмятане на едни и същи изрази – потенциален проблем, който има добри теоретични и практически решения
- използването му при решаване на задачи от процедурен (алгоритмичен) характер е неестествено и често неефективно

Основни компоненти на функционалните програми

Функционалните програми се изграждат чрез комбинация от изрази, функции и структури от данни. Основните градивни елементи в тях включват:

- Типове и типови декларации - всеки израз и функция имат тип. Haskell използва силна статична типизация, като типовите декларации улесняват разбирането и проверката на кода.
- Изрази - представят стойности или пресмятания. Всеки израз има стойност и може да бъде друг израз, стойност или функция. Изразите могат да бъдат примитивни или съставни (комбинация от подизрази).
- Функции - централният компонент във функционалното програмиране. Функциите са стойности от първи ред, което означава, че могат да бъдат предавани като аргументи, връщани като резултати и съхранявани в структури. Дефинират се чрез посочване на името, параметрите и израза, който указва какво връща функцията.
- Композиция на функции - изграждането на по-сложни функции чрез комбиниране на съществуващи. Използва се операторът (.) за свързване на функции така, че резултатът от една да е вход на следващата.
- Структури от данни - използват се за представяне на съставна информация. Най-често използваните са списъци ([a]), наредени двойки и тройки (кортежи), булеви стойности (True, False), числови типове (Int, Float), символи (Char) и дефинирани от потребителя типове.
- Абстракция и рекурсия - функционалните програми използват абстракция чрез дефиниране на обобщени функции, както и рекурсия за повторение, тъй като не се използват традиционни цикли.

В контекста на Haskell и функционалното програмиране, **примитивен израз** наричаме такъв израз, който не може да бъде разложен на по-прости съставни части. Това са най-базовите елементи на езика, които се оценяват директно до стойност без нужда от допълнително пресмятане или извикване на функции.

Примери:

```
1 [] // празният списък
2 True False // булеви стойности
```

```

3 42 3.14 // числови литерали
4 'a' 'z' // символни литерали
5 not abs // имена на функции без аргументи

```

Тези примитивни изрази служат като основа за изграждане на по-сложни функционални конструкции чрез комбиниране, вложени извиквания и абстракции.

Средства за комбиниране и абстракция

Функционалното програмиране се отличава с мощни средства за комбиниране на функции и изрази, както и за изграждане на абстракции, които позволяват кратък, ясен и повторно използваем код. Средствата за комбиниране и абстракция в Haskell позволяват изграждане на програми чрез декларативно изразяване на зависимостите, а не чрез императивно описание на стъпки. Те допринасят за краткия, модулен и лесен за проверка код - ключови цели на функционалното програмиране. Основните техники включват:

- Композиция на функции (function composition) - свързване на две или повече функции така, че резултатът от едната да бъде подаден като аргумент на следващата. Това позволява изграждане на нови функции чрез комбиниране на вече съществуващи, без нужда от междинни променливи. Пример:

```

1 (f . g) x = f (g x)

```

- Кариране (currying) - процес, при който функция с няколко аргумента се преобразува в последователност от едноаргументни функции. Това позволява частично прилагане на функции, като фиксираме само част от аргументите и получаваме нова функция. Пример:

```

1 f :: Int -> Int -> Int
2 f x y = x + y
3 // еквивалентно на
4 f :: Int -> (Int -> Int)
5 f x = g where g y = x + y

```

- Анонимни функции (lambda expressions) - използват се за създаване на функции без име - обикновено когато са необходими за кратко, например като аргумент на друга функция. Пример:

```

1 \x -> x + 1

```

- Функции от по-висок ред (higher order functions) - функции, които приемат други функции като аргументи или връщат функции като резултат. Пример:

```

1 map (\x -> x * 2) [1,2,3] // [2,4,6]

```

- Let-изрази и локални дефиниции - Позволяват въвеждане на локални помощни функции или стойности. Пример:

```
1 let square x = x * x in square 5
```

Оценяването на израз представлява процесът на изчисление, чрез който даден израз се свежда до стойност. Във функционалното програмиране изразите са основна единица на изчисление и оценяването им се извършва чрез последователно прилагане на функции, операции и подмяна на символи със съответните стойности или дефиниции. Всеки израз има стойност, която може да бъде примитивна (число, символ, булева стойност) или сложно (друга функция или структура от данни). Процесът на оценяване следва строго дефинирани правила, които определят как и в какъв ред се извършва редукцията на изразите. Независимо от конкретния език или модел на оценка, крайната цел е винаги да се получи стойност, до която се свежда даденият израз. Пример:

```
1 2 + 3           // оценява се до 5
2 (1 + 2) * 4     // първо 1 + 2 → 3, после 3 * 4 → 12
3 abs (negate 5) // negate 5 → -5, abs (-5) → 5
```

Всяка функционална програма представлява извикване на основна функция, която пресмята крайният резултат чрез множество помощни функции. Описанията на функциите включват декларация и **дефиниция**. Декларациите показват връзката между дадено име (идентификатор) и стойност от съответен тип. В най-простия случай те имат вида:

```
1 f :: type
```

Символът "::**type**" се чете като "е от тип"където **f** е идентификатора, а **type** - типът на връщаната стойност. В Haskell имената (идентификаторите) на функциите и останалите величини започват с малки букви, докато имената на типовете започват с главни. Дефинирането на функция с параметри има следния вид:

```
1 f x1 x2 ... xk = expression
```

Тук **x1**, **x2**, ..., **xk** са формални параметри – имената на независимите променливи, чрез които се изчислява резултата, а **expression** е израз, който указва правилото за пресмятане (намиране) на връщаната стойност. Изразите се образуват чрез композиция на операции (други функции) над параметрите. По-общо, дефиницията на функция може да включва поредица от изрази от посочения вид, с помощта на които се описват отделните случаи на действието на функцията. В тях съответните **x1**, **x2**, ..., **xk** са образци. Както видяхме по-горе дефиницията на функцията трябва да бъде предшествана от декларация на типа. В общия случай тя има вида:

```
1 f :: t1 -> t2 -> ... -> tk -> type
```

където `t1`, ..., `tk` са типовете на съответните параметри. Типовете в Haskell могат да бъдат примитивни (атомарни) и структурни. Към първите спадат булевите константи (`True/False`), числата (цели и реални) и знаковете, а към вторите - векторите (tuples) и списъците (включително низовете, които са списъци от знакове). Пример:

```
1 f :: Int -> Int -> Int
2 f x y = x + y
```

Когато дадена функция бъде приложена към аргументи, се извършва процес на **оценяване**, при който се подменят формалните параметри с подадените реални стойности. Този процес води до изчисляване на стойността на израза в тялото на функцията. Например, при извикване `f 3 4` се извършва следната редукция $f\ 3\ 4 \rightarrow 3 + 4 = 7$. Оценка на приложение на функция следва реда, наложен от семантиката на езика – първо се проверява дефиницията на функцията, след това се пресмятат (или отлагат) аргументите и накрая се извършва заместване и изчисление.

Във функционалното програмиране съществуват два основни **модела на оценяване**, които определят реда и стратегията за пресмятане на аргументи - **апликативно (стриктно, call-by-value)** и **нормално (лениво, call-by-name)**. При апликативното оценяване първо се пресмятат (оценяват) аргументите, и едва след това резултатите от тези изчисления се подават на функцията. Това е т.н. strict evaluation, тъй като изразите се оценяват, независимо дали стойностите им ще бъдат използвани в тялото на функцията. Този модел на оценка е характерен за езици като Scheme, където всяко аргументно изразяване се извършва преди извикването на функцията. Пример:

```
1 (define (f x y) x)
2 (f 10 (/ 1 0)) // Грешка: деление на нула
```

При нормалното оценяване аргументите не се оценяват веднага, а само когато – и ако – е необходимо. Това е модел на отложено оценяване, наричан още lazy evaluation, и позволява дефиниране на безкрайни структури и избягване на ненужно изчисление. Haskell използва този модел по подразбиране. Аргументите към функцията се предават като неизчислени изрази (thunks), и се оценяват едва когато стойността им е необходима в тялото на функцията. Пример:

```
1 f x y = x
2 f 10 (1 `div` 0) // Резултат: 10
```

2. Функции от по-висок ред. Анонимни (лямбда) функции.

Функция от по-висок ред се нарича всяка функция, която получава поне една функция като параметър или връща функция като резултат.

Примери:

- map - прилага дадена функция върху всеки елемент на списък

```
1 // map f xs = [f x | x <- xs]
2 map (+1) [1,2,3] // Оценка: [2, 3, 4]
```

- concatMap - прилага функция върху всеки елемент от списък и конкатенира получените списъци

```
1 concatMap (replicate 2) [1,2] // Оценка: [1, 1, 2, 2]
```

- filter - избира елементи от списък, които удовлетворяват дадено условие

```
1 // filter p xs = [x | x <- xs, p x]
2 filter even [1,2,3,4] // Оценка: [2, 4]
```

- zip - трансформира два списъка в списък от двойки от съответните елементи на двата списъка

```
1 zip [1, 2, 3, 4] [5, 6, 7] // Оценка: [(1, 5), (2, 6),
2 // (3, 7)]
```

- zipWith - комбинира два списъка елемент по елемент с дадена функция

```
1 zipWith (+) [1,2,3] [4,5,6] // Оценка: [5, 7, 9]
```

- foldl - рекурсивно прилага функция за натрупване отляво надясно

```
1 foldl (+) 0 [1,2,3] // Оценка: 6
2 foldl1 (-) [1,2,3] // Оценка: -4
```

- foldr - рекурсивно прилага функция за натрупване отдясно наляво

```
1 foldr (*) 1 [1,2,3] // Оценка: 6
2 foldr1 (/) [1,2,3] // Оценка: 1.5
```

- takeWhile - взима елементи от списък, докато дадено условие е вярно

```
1 takeWhile (< 5) [1, 2, 4, 6, 3] // Оценка: [1, 2, 4]
```

- dropWhile - пропуска елементи от списък, докато дадено условие е вярно

```
1 dropWhile (< 5) [1, 2, 4, 6, 3] // Оценка: [6, 3]
```

- all - проверява дали всички елементи на списък удовлетворяват дадено условие

```
1 all even [2, 4, 6] // Оценка: True
```

- any - проверява дали поне един елемент на списък удовлетворява дадено условие

```
1 any (> 5) [1, 2, 7, 3] // Оценка: True
```

Във функционалното програмиране функциите могат да бъдат предавани като аргументи на други функции, което ги прави напълно равноправни на останалите стойности в езика. Това означава, че всяка функция може да получава друга функция като параметър и да я използва в своето тяло. Това поведение е в основата на функциите от по-висок ред и позволява изграждане на гъвкави и обобщени изчисления. Пример:

```
1 applyTwice f x = f (f x)
2 applyTwice (*2) 3 // Оценка: 12
```

Функцията `applyTwice` приема функция `f` като параметър и я прилага два пъти върху стойността `x`.

Функциите могат да се подават и като резултат от други функции. Така наречените **оценки на обръщения към функции** се отнасят до ситуации, в които извикването на функция връща друга функция като резултат, която може впоследствие да бъде приложена. Пример:

```
1 add x y = x + y
2 (add 3) 5 // Оценка: 8
```

В този случай `add 3` връща функция, която прибавя 3 към подадения ѝ аргумент. Това показва, че резултатът от едно обръщение към функция може да бъде нова функция, която също подлежи на оценка.

Ламбда нотацията се използва, когато имаме за цел да използваме директно дадена функция, вместо да я именуваме и дефинираме. В Haskell този вид изрази се наричат **ламбда изрази**, а функциите дефинирани чрез тях се наричат **анонимни функции**.

Пример:

```
1 addNumber n = \m -> n + m

1 map (\x -> x * 2) [1,2,3] // Оценка: [2, 4, 6]
2
3 filter (\x -> x `mod` 2 == 0) [1,2,3,4,5] // Оценка: [2, 4]
```

Ламбда функциите се оценяват в момента на изпълнение, тъй като не се именува. Те често се използват като аргументи на функции от по-висок ред като `map`, `filter`, `foldr` и др.

20. Функционално програмиране. Списъци. Потоци и отложено оценяване.

1. Списъци. Представяне. Основни операции.

Най-често използваната структура във функционалните езици е **списъкът**. Той представя редица от (променлив брой) елементи, принадлежащи на един и същ тип. Записът на списъците в Haskell е следният:

- `[]` обозначава празен списък (списък без елементи)
- `[e1...en]` обозначава списък с елементи $e_1 \dots e_n$

Друга форма на запис на списъци от числа, знакове и елементи на изброими типове е чрез аритметична последователност:

```
1 [n..m] // е списъкът [n, n + 1, ..., m]
2 [n, p..m] // е списъкът, чийто първи два елемента са n и p,
3 // последният му елемент е m и стъпката на нарастване на елементите
4 // му е p - n
```

Конструирането на списъци чрез определяне на техния обхват (List Comprehension) ни дава по-голяма свобода. Синтаксисът е следният:

```
1 [expr | q1, ..., qk]
```

където `expr` е израз, а q_i може да бъде генератор от вида $p \leftarrow lExpr$ или филтър, който е булев израз.

Примери:

```
1 [2*n | n <- [2, 4, 7]] // е списъкът [4, 8, 14]
2 [isEven n | n <- [2, 4, 7]] // е списъкът [True, True, False]
3 [2*n | n <- [2, 4, 7], isEven n, n > 3] // е списъкът [8]
```

Основни операции със списъци:

- Проверка за празен списък

```
1 null [] // True
2 null [1,2] // False
```

- Конкатенация

```
1 [1,2] ++ [3,4] // [1,2,3,4]
```

- Дължина

```
1 length [1,2,3] // 3
```

- Принадлежност на елемент към списък

```
1 elem 3 [1,2,3] // True
```

- Достъп до главата на списъка

```
1 head [1,2,3] // 1
```

- Достъп до опашката на списъка

```
1 tail [1,2,3] // [2,3]
```

- Достъп до елемент по индекс

```
1 [1, 2, 3] !! 1 // 2
```

- Селектиране на елементи

```
1 take 2 [1,2,3,4] // [1,2]
```

```
2 drop 2 [1,2,3,4] // [3,4]
```

- Обръщане на списък

```
1 reverse [1,2,3] // [3,2,1]
```

Функции от по-висок ред за работа със списъци:

- map - прилага функция към всеки елемент от списък

```
1 map (*2) [1,2,3] // [2,4,6]
```

- filter - избира тези елементи от списък, които удовлетворяват дадено условие

```
1 filter even [1,2,3,4] // [2,4]
```

- foldr / foldl - агрегира(свива) списък до една стойност отдясно наляво / отляво надясно

```
1 foldr (+) 0 [1,2,3] // 6
```

```
2 foldl (*) 1 [1,2,3] // 6
```

- zip - слива два списъка в списък от наредени двойки

```
1 zip [1,2] [3,4] // [(1,3),(2,4)]
```

- unzip - превръща списък от наредени двойки в наредена двойка от списъци

```
1 unzip [(1,3),(2,4)] // ([1,2],[3,4])
```

2.Безкрайни потоци и безкрайни списъци. Отложено оценяване.

Във функционалното програмиране, и особено в Haskell, е възможно да се дефинират безкрайни структури от данни, благодарение на отложено оценяване (lazy evaluation). Това означава, че елементите се изчисляват само когато бъдат нужни. В Haskell безкрайните потоци всъщност са обикновени списъци, чиято дължина може да бъде безкрайна благодарение на отложеното оценяване.

Примери:

```
1 stream n = n : stream (n + 1) // безраен поток от естествени
2 // числа
3 naturals = [1..] // безкраен списък от естествени числа
```

Потокът `stream` и списъкът `naturals` не се изчисляват наведнъж, но можем да вземем например началните им елементи:

```
1 take 5 (stream 1) // [1, 2, 3, 4, 5]
2 take 5 naturals // [1, 2, 3, 4, 5]
```

Основните операции и функциите от по-висок ред върху безкрайни списъци използваме по същия начин, като при обикновените списъци.

Примери:

```
1 null naturals // False
2 elem 3 naturals // True
3 head naturals // 1
4 tail naturals // [2, 3, 4, 5, 6, ...]
5 take 5 naturals // [1, 2, 3, 4, 5]
6 drop 5 naturals // [1, 2, 3, 4, 5]
7 map (*2) (take 5 naturals) // [2, 4, 6, 8, 10]
8 filter even (take 10 naturals)
9 foldr (+) 0 (take 5 naturals) // 1 + 2 + 3 + 4 + 5 = 15
10 zip (take 5 naturals) ['a'..] // [(1,'a'), (2,'b'), (3,'c'),
11 // (4,'d'), (5,'e')]
12 unzip (zip (take 3 naturals) ['x', 'y', 'z'])
13 // ([1, 2, 3], ['x', 'y', 'z'])
```

Има и някои основни операции, които не са приложими върху безкрайни списъци:

```
1 foldl/foldl1 naturals // натрупваща операция отляво, няма да
2 // завърши никога, защото трябва да обработи целия списък
3 length naturals // никога не завършва, защото дължината е ∞
4 sum naturals
5 product naturals
```

6 last naturals

Отложеното оценяване (lazy evaluation) е стратегия на оценяване, която по стандарт стои в основата на работата на всички интерпретатори на Haskell. Същността на тази стратегия е, че интерпретаторът оценява даден аргумент на дадена функция само ако стойността на този аргумент е необходима за пресмятането на целия резултат. Ако даден аргумент е съставен (например вектор или списък), то се оценяват само тези негови компоненти, чиито стойности са необходими за получаването на резултата. При това дублиращите се подизрази се оценяват по не повече от един път. Когато се оценява израз - обръщение към функцията, се оценява частен случай на тялото на дефиницията, в който формалните параметри са заместени с подадени аргументи.

Пример:

```
1 f x y = x + y
2 ...
3 f (9 - 3) (f 34 3) // се оценява до (9 - 3) + (f 34 3)
```

Изразите не се оценяват преди да има реална нужда от техните стойности. В конкретния случай ще се наложи оценка и на двата фактически параметъра, но това не винаги е така.

Пример:

```
1 g x y = x + 5
2 ...
3 g (9 - 3) (g 34 3) // се оценява до (9 - 3) + 5
```

Това е едно от най-важните преимущества на отложеното оценяване - ненужните аргументи не се оценяват.

Пример:

```
1 switch n x y
2 | (n > 0) = x
3 | otherwise y
4 // Тук при всяко извикване ще се оценяват стойностите на
5 // параметрите n и само един от изразите x и y
```

Ако $\text{rm}(x, y) = x + 1$, $\text{rm}(3 + 2, 4 - 17)$ се оценява до $(3 + 2) + 1$. Макар че аргументът е вектор, отново се оценява само частта нужна за пресмятане на резултата.

Когато извикваме функция и се опитваме да съпоставим образец, аргументите трябва да се оценят с цел да се установи кое от условните равенства

е приложимо. Оценяването на аргументите обаче не се извършва изцяло, а само до степен, която е достатъчна, за да се прецени дали те са съпоставими със съответните образци. Когато се намери подходящо равенство, оценяването продължава с директното му прилагане. Едно важно следствие от отложеното оценяване в Haskell е обстоятелството, че езикът позволява да се работи с безкрайни структури. Пълното оценяване на такава структура изисква безкрайно време, т.е. не може да завърши, но механизмът на отложеното оценяване позволява да бъдат оценявани само тези части ("порции") на безкрайните структури, които са необходими.

Логическо програмиране

21. Синтаксис и семантика на предикатното смятане от първи ред.

Дефиниция Език на предикатното смятане от първи ред дефинираме чрез:

- Логическа част - азбука на индивидните променливи $\{x, y, z, \dots\}$, азбука на съжителните връзки $\{v, \&, \implies, \iff\}$, азбука на кванторите $\{\exists, \forall\}$, азбука на помощните символи $\{., (,)\}$, формално равенство (опционално) $\doteq$
- Нелогическа част - азбука на индивидните константи $Const_{\mathcal{L}}$, азбука на предикатните символи $Pred_{\mathcal{L}}$, азбука на функционалните символи $Func_{\mathcal{L}}$, функция на арност $\# : Func_{\mathcal{L}} \cup Pred_{\mathcal{L}} \rightarrow \mathbb{N} \setminus \{0\}$

Нека sig е сигнатура.

Дефиниция Терм при сигнатура sig дефинираме индуктивно:

- ако x е индивидна променлива, то x е sig -терм
- ако c е символ за константа от sig , то c е sig -терм
- ако f е n -местен функционален символ от sig и $\tau_1, \dots, \tau_n$ са sig -термове, то низът $f(\tau_1 \dots \tau_n)$ е sig -терм.

Дефиниция Ако p е n -местен предикатен символ от sig и $\tau_1, \dots, \tau_n$ са sig -термове, то низът $p(\tau_1, \dots, \tau_n)$ е атомарна формула при сигнатура sig .

Дефиниция Предикатна формула при сигнатура sig дефинираме индуктивно:

- ако φ е атомарна sig -формула, то φ е sig -формула
- $\perp$ е sig -формула
- ако φ и ψ са sig -формули, то низовете $(\varphi \& \psi)$, $(\varphi \vee \psi)$ и $(\varphi \implies \psi)$ са sig -формули

- ако x е променлива и φ е sig -формула, то низовете $\forall x\varphi$ и $\exists x\varphi$ са sig -формули

Дефиниция Казваме, че ψ е подформула на φ , ако формулата ψ е част от формулата φ .

Дефиниция Нека φ е формула, която съдържа подформула от вида $\forall x\psi$ или $\exists x\psi$. Казваме, че тази подформула е **областта на действие** на квантора $\forall x$ или $\exists x$, с който започва подформулата.

Дефиниция Едно срещане на променливата x наричаме **свързано**, ако попада в областта на действие на някой квантор $\forall x$ или $\exists x$ и **свободно**, ако не попада в областта на действие на никой квантор $\forall x$ или $\exists x$.

Дефиниция **Затворена формула** наричаме формула, която не съдържа свободни променливи.

Дефиниция Нека $sig = \langle d_1, d_2, \dots \rangle$ е сигнатура. **Структура** за sig наричаме редицата $M = \langle [[индив]]^M, [[d_1]]^M, [[d_2]]^M, \dots \rangle$ ако $[индив]^M \neq \emptyset$ наречено универсум. Освен това:

- ако d_i е символ за константа, то интерпретацията $[[d_i]]^M$ на d_i е елемент на универсума
- ако d е n -местен функционален символ, то интерпретацията $[[d_i]]^M$ на d_i е n -местна функция в универсума
- ако d е n -местен предикатен символ, то интерпретацията $[[d_i]]^M$ на d_i е n -местна релация в универсума

Дефиниция Функцията v е **оценка** в структурата M , ако v съпоставя на всяка променлива елемент на универсума на M .

Дефиниция Нека M е структура. За всяка формула φ от сигнатурата на M и оценка v в M ще дефинираме съждение, което ще записваме $M \models \varphi[v]$ и ще четем така: "Формулата φ е вярна в структурата M при оценка v ."

Дефиниция Формулата φ е (**тъждествено**) **вярна** в структура M , ако е вярна в структурата M при произволна оценка v ($M \models \varphi$). Формулата φ е тъждествено вярна или **предикатна тавтология**, ако φ е тъждествено вярна във всяка структура ($\models \varphi$).

Дефиниция **Субституция** наричаме функция, която на всяка променлива съпоставя терм.

Твърдение

Нека v и v' са оценки в структура M . Ако v и v' съвпадат за всички свободни променливи на формулата φ , то $M \models \varphi[v]$ е еквивалентно на $M \models \varphi[v']$.

Лема (за субституциите на термове)

Нека $\tau(x_1, \dots, x_n)$ и $\sigma_1, \dots, \sigma_n$ са термове и v е оценка в структурата M . Тогава $[[\tau[\sigma_1, \dots, \sigma_n]]]^M v = [[\tau]]^M [[[\sigma_1]]^M v, \dots, [[\sigma_n]]^M v]$

22. Изводимост и компютърно генериране на доказателства.

Дефиниция Литерал наричаме атомарна формула или отрицание на атомарна формула.

Дефиниция Дизюнкт наричаме крайно множество от литерали. Празният дизюнкт означаваме с $\blacksquare$.

- Един дизюнкт е **неверен** в структура M при оценка v , ако всичките му литерали са неверни в M при оценка v .
- Един дизюнкт е **верен в структура** M при оценка v , ако не е вярно, че е неверен в M при оценка v .
- Един дизюнкт е (тъждествено) **верен в структура** M , ако е верен при всяка оценка в M .
- Множество от дизюнкти е **изпълнимо**, ако съществува структура M , в която са тъждествено верни всички дизюнкти от множеството. Структурата M е модел на това множество.
- Множество от дизюнкти е **неизпълнимо**, ако не е вярно, че то е изпълнимо (т.е. ако няма модел).

Дефиниция Нека $\{\varphi, \lambda'_1, \dots, \lambda'_n\}$ и $\neg\varphi, \lambda''_1, \dots, \lambda''_k$ са дизюнкти. Дизюнкта $\{\lambda'_1, \dots, \lambda'_n, \lambda''_1, \dots, \lambda''_k\}$ наричаме тяхна непосредствена **резолвента**. **Либерална резолвента** или просто **резолвента** на два дизюнкта наричаме непосредствена резолвента на кои да е частни случаи на тези дизюнкти.

Дефиниция Нека Γ е множество от дизюнкти. **Резолютивен извод** на дизюнкта δ от Γ наричаме редица от дизюнкти, така че $\delta_0, \dots, \delta_n$ в която $\delta_n = \delta$ и за всеки от дизюнктите в редицата е вярно поне едно от условията:

- че е елемент на Γ
- че е резолвента на дизюнкти, които се срещат по-рано в редицата

Дизюнктът δ се извежда от Γ с резолюция ($\Gamma \vdash \delta$), ако съществува резолютивен извод на δ от Γ .

Твърдение 1

Празният дизюнкт не е тъждествено верен в никоя структура.

Твърдение 2

Всяка резолвента на дизюнкти, които са верни в структура M , е вярна в M .

Теорема (коректност на резолютивната изводимост)

Нека Γ е множество от дизюнкти. Ако Γ е изпълнимо, то $\blacksquare$ не е изводим от Γ с либерална резолюция.

Доказателство. Да допуснем, че дизюнктите от Γ имат модел M , но въпреки това $\blacksquare$ не е изводим от Γ . Нека $\delta_1, \dots, \delta_n$ е произволен резолютивен извод на $\blacksquare$ от Γ . Съгласно **Твърдение1**, $\blacksquare$ е неверен в M , а $\delta_n = \blacksquare$, то резолютивният извод $\delta_1, \dots, \delta_n$ съдържа поне един дизюнкт, който е неверен в M . Нека j е най-малкото число, за което δ_j е неверен в M . От дефиницията за резолютивен извод, за дизюнкта δ_j има две възможности. Първата е той да бъде елемент на Γ . В този случай знаем, че той е верен в M и това е противоречие. Втората възможност е δ_j да бъде резолвента на предходни дизюнкти в редицата. Тъй като j бе най-малкото число, за което δ_j е неверен в M , ти тези предходни дизюнкти са верни в M и значи съгласно **Твърдение2**, резолвентата δ_j също е вярна в M . Така и в този случай получаваме противоречие. $\square$

Теорема (пълнота на резолютивната изводимост)

Нека Γ е множество от дизюнкти. Ако не е вярно $\Gamma \vdash \blacksquare$, то Γ има модел.

Бази от данни

23. Бази от данни. Релационен модел на данните.

В основата на **релационния модел** стои математическото понятие n -членна релация. Всяка релация е множество от елементи, които се състоят от n компоненти и се наричат n -торки. Предимства на релационния модел са:

- Простота на модела - възможност за опериране с релациите както с числата, като се използват операциите (релационна алгебра)
- Висока степен на независимост на данните
- Съответства на начина на човешкото мислене
- Еднообразен начин за представяне на данните и връзките между тях във вид на таблица

Дефиниция Домейн наричаме множество от допустими стойности за дадена величина.

Пример: Домейнът на атрибута "age" може да е множеството от цели числа между 0 и 110.

Дефиниция Релация наричаме всяко подмножество на декартовото произведение $D_1 \times \dots \times D_n$.

Пример: Релация "Students" с атрибути `name`, `age` и `studentID`.

Дефиниция Кортежи наричаме множеството от атрибутите на дадена релация.

Пример: ('Ivan', 20, '12345') е кортеж в релацията "Students".

Дефиниция Атрибути наричаме имената (означенията) на колоните на дадена релация.

Пример: `name`, `age`, `studentID` са атрибути.

Дефиниция Схема на релация наричаме името на релацията и множеството от атрибутите ѝ.

Пример: `Students(name, age, studentID)`.

Дефиниция Схема на релационна база от данни наричаме множеството от всички схеми на релации в базата от данни.

Реализация на релационна база от данни

Процесът на проектиране на една релационна база от данни се състои от следните стъпки:

- Определяне на данните (обектите и техните характеристики), които ще се съхраняват
- Определяне на връзките между обектите
- Определяне на ключовите и на свързващите колони
- Определяне на ограниченията върху обектите и връзките между тях
- Отстраняване на евентуални недостатъци (излишества и пр.)
- Реализиране на базата от данни

Дефинирането на отделните релационни схеми е твърде свободно, но най-често при създаването им се спазват следните две правила:

- Всеки клас от обекти се представя с релация, чиято схема включва всички негови атрибути, които стават атрибути и на релацията. Всяка n-торка от релацията представя конкретен обект/екземпляр от класа. Ключовият атрибут или списъкът от ключовите атрибути на класа от обекти, т.е. тези, които еднозначно идентифицират отделните екземпляри в класа, се приемат за ключ на релацията.
- Връзките между два или повече класове от обекти се представят чрез релация, чиято релационна схема включва ключовите атрибути на всеки от тези класове от обекти.

Видове операции върху релационната база от данни.

Заявки към релационната база от данни

Възможни са два вида операции върху релационната база от данни:

- Операции с данни - осъществяват се чрез DML (Data Manipulation Language). Те включват **SELECT** - извличане на данни, **INSERT** - добавяне на нови редове, **UPDATE** - промяна на съществуващи стойности и **DELETE** - изтриване на редове. Тези операции не засягат структурата на таблицата, а само самите данни (кортежите). За да станат направените промени постоянни, след някои DML операции (в зависимост от системата) се прави **COMMIT**. Примери:

```
1 SELECT * FROM Students;
2
3 INSERT INTO Students (name, age, studentID) VALUES
   ('Georgi', 22, 12346);
4
5 UPDATE Students SET age = 23 WHERE name = 'Georgi';
6
7 DELETE FROM Students WHERE WHERE age < 21;
```

- Операции със структура - осъществяват се чрез DDL (Data Definition Language). Те включват **CREATE** - създаване на нова таблица или друг обект, **ALTER** - промяна на вече съществуваща таблица и **DROP** - изтриване на таблица или друг обект. Тези операции служат за дефиниране и промяна на структурата на таблиците и другите обекти в базата от данни (редове, колонии, индекси и т.н.). Когато се изпълни един DDL statement, промените се прилагат незабавно. Примери:

```
1 CREATE TABLE Students (
2     studentID INTEGER PRIMARY KEY,
3     name VARCHAR(50) NOT NULL,
4     age INTEGER
5 );
6
7 ALTER TABLE Students ADD email VARCHAR(100);
8
9 ALTER TABLE Students DROP COLUMN email;
10
11 DROP TABLE Students;
```

Дефиниция Релационна алгебра наричаме алгебра, чиито операнди са релации или променливи, които представят релации. Тя е в основата на всеки език за заявки към база от данни - всички системи за управление на бази от данни използват релационната алгебра като междинен език за изчисление на заявките.

Основни операции

1. Обединение - $R \cup S$

- Връща всички кортежи от двете релации без повторения
- Бинарна, комутативна, асоциативна
- Пример с SQL:

```
1 SELECT name FROM Students UNION SELECT title FROM
2 Courses;
```

2. Разлика - $R - S$

- Връща всички кортежи от релацията R , които не се съдържат в релацията S
- Бинарна, некомутативна, неасоциативна
- Пример с SQL:

```
1 SELECT name FROM Students EXCEPT SELECT title FROM
2 Courses;
```

3. Декартово произведение - $R \times S$

- Множеството от всички двойки, при които първият елемент е произволен от R , а вторият от S
- Бинарна, комутативна, асоциативна
- Пример с SQL:

```
1 SELECT * FROM Students CROSS JOIN Courses;
```

4. Проекция

- $\pi_{\langle attrlist \rangle}(R)$, където $\langle attrlist \rangle$ е списък с атрибути
- Унарна
- Пример с SQL:

```
1 SELECT name, age FROM Students;
```

5. Селекция

- $\sigma_{\langle predicate \rangle}(R)$, където $\langle predicate \rangle := \langle attr \rangle \langle op \rangle \langle attr | const \rangle$ и $op \in \{=, \neq, <, >, \leq, AND, OR, \dots\}$
- Унарна, комутативна (при композиция на селекции)
- Пример с SQL:

```
1 SELECT * FROM Students WHERE age > 20;
```

Допълнителни операции

1. Сечение - $R \cap S$

- Връща общите кортежи между R и S
- Бинарна, комутативна, асоциативна
- Пример с SQL:

```
1 SELECT studentID FROM Students INTERSECT SELECT
2   courseID FROM Courses;
```

2. Частно - $R : S$

- Връща тези кортежи от R, които са свързани с всички елементи от S
- Бинарна, некомутативна, неасоциативна
- Частното на R и S е трета релация, която има свойството, че $\forall t \in R$ и $\forall s \in S$, конкатенацията $ts \in P$

3. Съединение - $R \bowtie_C S$, където C е условието на свързване

- Декартово произведение на R и S и избор на кортежите, които удовлетворяват условието C
- Пример с SQL:

```
1 SELECT * FROM Students JOIN Courses ON
2   Students.studentID = Courses.ID;
```

4. Естествено съединение - $R \bowtie S$

- Автоматично свързва по всички атрибути с еднакви имена и премахва дублираните колони
- Пример с SQL:

```
1 SELECT * FROM Students NATURAL JOIN Courses;
```

24. Бази от данни. Нормални форми.

Нормализацията на релационна база от данни е процесът на реструктуриране на релациите и зависимостите на техните атрибути в нормални форми, които целят минимизация на дублирането на данните, както и запазването на тяхната цялост.

Проектирането на схема на релационна база от данни представлява процес на преобразуване на концептуалния модел (например модел

"същност-връзка") в съвкупност от релации, върху чиито атрибути са дефинирани зависимости. Целта на този процес е да се създаде подходяща структура на базата от данни, така че:

- да се запази цялостта на данните
- да се нормализират получените релации

Аномалии, ограничения, ключове

Дефиниция Аномалия в базите от данни е нежелано поведение или непоследователност, която възниква при вмъкване, изтриване или актуализация на данни, поради лоша структура на релационната схема.

При проектиране на релационните схеми, могат да бъдат получени следните аномалии:

- Излишества - повторение на информация без това да бъде необходимо
- Аномалии при добавяне - получава се когато се опитваме да добавим информация, но тя не може да бъде добавена освен по половинчат начин. Случва се при релации с транзитивни функционални зависимости
- Аномалии при обновяване - получава се когато при промяна на данни в един кортеж не се обновят свързаните с него кортежи и се стигне до неконсистентна информация в следствие на прекъсната заявка
- Аномалии при изтриване - получава се когато се опита да изтрием информация, но това не може да се случи без премахването на друга важна информация. Случва се при релации с транзитивни функционални зависимости

Дефиниция Ограниченията са механизъм за налагане на цялост върху данните чрез дефиниране на правила, на които трябва да отговарят стойностите на конкретните атрибути (не позволяват въвеждането на невалидни данни). Основните ограничения са:

- Уникалност на множество от атрибути - два кортежа не могат да имат едни и същи стойности на атрибутите от множеството. По дефиниция първичните ключове са уникални. В *SQL* се имплементира чрез ключовите думи **UNIQUE** и **PRIMARY_KEY**.
- Задаване на домейн от възможни стойности на множество от атрибути - имплементират се чрез предикати. В *SQL* обичайно се имплементира чрез ключовите думи **CHECK** и **NOT NULL**.
- Външен ключ - представлява множество от атрибути, такова че за всяка ненулева стойности на атрибут, който е външен ключ, трябва да съществува ключов атрибут от друга релация, който да съдържа

същата стойност. В *SQL* обичайно се имплементира чрез ключовата дума **FOREIGN_KEY**.

Дефиниция Множеството $\{A_1, \dots, A_n\}$ наричаме **ключ** на R , ако:

- $\{A_1, \dots, A_n\}$ функционално определя всички останали атрибути на R
- $\{A_1, \dots, A_n\}$ е минимално (не съществува подмножество на $\{A_1, \dots, A_n\}$ което също функционално определя всички останали атрибути на R)

Дефиниция **Суперключ** наричаме множество от атрибути, което съдържа ключа.

Функционални зависимости

Нека $X = \{A_1, \dots, A_n\}$ и $Y = \{B_1, \dots, B_m\}$ са множества от атрибути на релация R .

Дефиниция Казваме, че $X \rightarrow Y$ е **функционална зависимост** (X функционално определя Y), ако $\forall t, u \in R$ е изпълнено: ако $t[X] = u[X]$, то $t[Y] = u[Y]$. Функционалната зависимост $X \rightarrow Y$ означава, че стойностите на атрибутите в X еднозначно определят стойностите на атрибутите в Y .

Пример: $\text{studentID} \rightarrow \text{name, age}$

Аксиоми (Армстронг / функционални зависимости)

1. Рефлексивност: ако $Y \subseteq X$, то $X \rightarrow Y$
2. Разширение: ако $X \rightarrow Y$, то $XW \rightarrow YW$
3. Транзитивност: ако $X \rightarrow Y$ и $Y \rightarrow Z$, то $X \rightarrow Z$

Нормализацията на база от данни е процес, насочен към преобразуването на релационни схеми, при който се налагат ограничения към новополучените релационни схеми, елиминиращи някои нежелани свойства.

Дефиниция Релацията R е в **1NF**, ако всеки атрибут във всеки кортеж има атомарна стойност.

Дефиниция Релацията R е във **2NF**, ако е в **1NF** и всеки неключов атрибут е в пълна функционална зависимост от ключа (зависи от целия ключ, а не от негово подмножество).

Дефиниция **Първичен атрибут** наричаме атрибут, който е част от ключа.

Дефиниция Релацията R е в **3NF**, ако за всяка ФЗ $X \rightarrow Y$ или X е суперключ, или всеки атрибут в Y е първичен.

Дефиниция Релацията R е в **BCNF**, ако за всяка нетривиална ФЗ $X \rightarrow Y$, X е суперключ за R .

Пример:

Нека имаме следната таблица

Student	Subject	Teacher
Ivan	Informatics	Prof. P. Petrov
Maria	Informatics	Prof. P. Petrov
Petar	Mathematics	Dr. I. Georgiev

Нормализация:

1. Таблицата има атомарни стойности, следователно е в $1NF$.
2. Ключът на таблицата е *Student*. Неключовите атрибути са *Subject*, който зависи от *Student* и удовлетворява изискванията за $2NF$. Обаче тук *Teacher* зависи от *Subject*, а не директно от *Student*. Това е проблем, защото преподавателят зависи от специалността. Сега ще разделим таблицата на две, за да нормализираме таблицата във $2NF$:

Student	Subject
Ivan	Informatics
Maria	Informatics
Petar	Mathematics

Subject	Teacher
Informatics	Prof. P. Petrov
Informatics	Prof. P. Petrov
Mathematics	Dr. I. Georgiev

3. Сега в таблица *Students*, студента определя директно специалността и в таблица *Subjects*, специалността определя директно преподавателя. Следователно няма транзитивни зависимости и е изпълнена $3NF$.
4. В таблица *Students*, студентът е ключ и в таблица *Subjects*, специалността е ключ, следователно сме в $BCNF$.

Многозначни зависимости

Терминът "многозначна зависимост" се използва, когато два атрибута или множество атрибути са независими помежду си. Тя обобщава идеята за ФЗ в този смисъл, че всяка ФЗ е частен случай на многофункционална зависимост. Съществуват схеми в **BCNF**, които също могат да съдържат излишни данни - най-често при моделиране на връзки "много към много" в една релация. Пример: в Tutor/Student Cross-Reference могат да се въведат два различни номера на социална осигуровка за един и същ ръководител, което е нежелателно.

Дефиниция Казваме, че $X \twoheadrightarrow Y$ е **многозначна зависимост** в R ако $\forall t, u \in R : t[X] = u[X] \Rightarrow \exists v \in R$ за който е изпълнено:

- $v[X] = t[X] = u[X]$
- $v[Y] = t[Y]$
- $v[Z] = u[Z]$, където $Z = Attr(R) \setminus (X \cup Y)$

Аксиоми (многозначни зависимости)

1. Транзитивност: ако $X \twoheadrightarrow Y$ и $Y \twoheadrightarrow Z$, то $X \twoheadrightarrow Z$
2. Допълнение: ако $X \twoheadrightarrow Y$, то $X \twoheadrightarrow (R \setminus (X \cup Y))$
3. Обединение: ако $X \twoheadrightarrow Y$ и $X \twoheadrightarrow Z$, то $X \twoheadrightarrow (Y \cup Z)$

Дефиниция Ако за многозначната зависимост $X \twoheadrightarrow Y$ е изпълнено, че $Y \not\subseteq X$ и $X \cup Y \neq Attr(R)$, то тя се нарича нетривиална.

Дефиниция Казваме, че разпределението на релационна схема R в подмножествата $R_1, \dots, R_n$ е **съединение без загуба**, ако естественото съединение на получените проекции възстановява оригиналната релация:

$$\pi(R_1) \bowtie \dots \bowtie \pi(R_n) = R$$

Дефиниция Релацията R е в **4NF**, ако за всяка нетривиална МЗ $X \twoheadrightarrow Y, X$ е суперключ.

Искусствен интелект

25. Търсене в пространството от състояния. Генетични алгоритми.

Решаването на много задачи, традиционно смятани за интелектуални, може да бъде сведено до последователно преминаване от една формулировка на задачата до друга, еквивалентна на първата или по-проста от нея, докато се стигне до това, което се смята за решение на задачата.

Дефиниция Състояние наричаме описание на задача в процеса на нейното решаване. Състоянията биват начални, междинни и крайни (цели).

Дефиниция Оператор наричаме правило (алгоритъм), по който от дадено състояние се получава друго.

Дефиниция Пространство на състоянията наричаме съвкупността от всички възможни състояния, които могат да се получат от дадено начално състояние. Представя се чрез ориентиран граф с възли - състоянията и дъги - операторите.

Основни типове задачи за търсене в ПС

1. **Търсене на път до определена цел** - търси се път от дадено начално състояние до определено целево състояние.

Според наличната информация:

- Неинформирано (сляпо) търсене - DFS, BFS, UCS, DLS, IDS
- Информирано (евристично) търсене - Best First Search, Beam Search, Hill Climbing, A*

Според разглежданото пространство:

- Глобално търсещи - DFS, BFS, UCS, IDS, Best First Search, A*
- Локално търсещи - Beam Search, Hill Climbing, Simulated Annealing, Genetic Algorithm

Характеристики на алгоритмите за търсене:

- Пълнота - ако съществува решение, алгоритъмът гарантира, че ще го намери
- Оптималност - ако има повече от един път до решение, алгоритъмът намира най-краткия (оптималния)
- Сложност по време - брой обходени възли
- Сложност по памет - максимален брой възли в паметта

Параметри на търсенето:

- d - дълбочина на най-близката цел
- m - максимална дълбочина на пространството от състояния
- b - коефициент на разклонение на графа на състоянията

2. **Формиране на стратегия при игри за двама играчи**

Разглеждат се т.н. интелектуални игри с пълна информация, които се играят от двама играчи и върху хода на които не оказват влияние случайни фактори. Двамата играчи играят последователно и всеки от тях има пълна информация за хода на играта. Най-често се решава задачата за намиране на най-добър (първи) ход на играча, който трябва да направи текущия ход.

Основни типове игри:

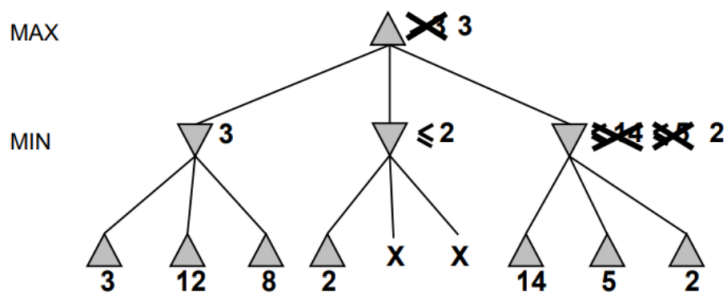
- с перфектна/неперфектна информация
- детерминистични/включващи елемент на шанс

- с пълна/непълна информация

Дървото на състоянията при тези задачи е дърво на възможните позиции в резултат на възможните ходове на двамата играчи. Общият брой възли в дървото на състоянията е от порядъка на b^d .

Алгоритми за решаване на задачи от този тип:

- Минимаксна процедура - метод за намиране на най-добрия ход на първия играч при предположение, че другият играч също играе оптимално. Предполага се, че двамата играчи имат противоположни интереси - единият търси стратегия, която води до получаване на позиция с максимална оценка (максимизиращ играч, MAX), а другият търси стратегия, която води до получаване на позиция с минимална оценка (минимизиращ играч, MIN). Минимаксната процедура е метод за получаване на оценките на възлите от по-горните нива на ДС, които позволяват на първия играч да избере най-добрия си ход.
- Минимаксна процедура с α - β отсичане - при α -отсичането не е необходимо да се извършва генериране и търсене върху всяко поддърво, което произлиза от минимизиращ възел, β -стойността на който е $\leq$ на α -стойността на съответния максимизиращ родител. При β -отсичането не е необходимо да се извършва генериране и търсене върху всяко поддърво, което произлиза от максимизиращ възел, α -стойността на който е $\geq$ на β -стойността на съответния минимизиращ родител.



3. Намиране на цел при спазване на ограничителни условия

Дадени са:

- Множество от променливи $v_1, \dots, v_n$ с области на допустимите стойности съответно Dv_i - дискретни (крайни или изброими безкрайни) или непрекъснати.
- Множество от ограничения $c_1, \dots, c_n$ (допустими/недопустими комбинации от стойности на променливите)

Видове ограничения:

- Унарни - включващи една променлива
- Бинарни - включващи две променливи
- От по-висок ред - включващи три или повече променливи
- Основани на предпочитания (меки) - показват коя стойност е по-добра за дадена променлива

Цел на задачата:

- Да се инициализират всички променливи със стойности от съответните им домейни по такъв начин, че и същевременно всички ограничения да бъдат удовлетворени
- Тоест целевото състояние е множество от свързвания със стойности на променливите $v_1 = c_1, \dots, v_n = c_n$, които удовлетворяват всички ограничения.

Алгоритми за решаване на задачи от този тип:

- Генериране и тестване
- Търсене с възврат (Backtracking)
- Разпространяване на ограничения (Constraint propagation) - Forward checking, Arc Consistency
- Локално търсещи - MinConflicts

Методи за информирано (евристично) търсене на път до определена цел

Реализират пълно изчерпване по гъвкава стратегия или търсене с отсичане на част от графа на състоянията. Приложими са при наличие на специфична информация за предметната област, позволяваща да се конструира оценяваща функция (евристика), която връща числова оценка (в предварително определен интервал). Тази оценка може да служи например за мярка на близост до целта или на необходимия ресурс за достигане ѝ. Най-често се използва евристична оценяваща функция h , която връща като резултат приблизителната стойност на определен ресурс, необходим за достигане от оценяваното състояние до целта.

Програмната реализация включва използване на работна памет (списък на т. н. открити възли или списък от изминати пътища, започващи от началния възел – фронт на търсенето). Възможните състояния, които следва да се обхождат се сортират въз основа на оценяваща функция, която определя кое състояние е по-добро (best-first - винаги вземаме най-доброто първо). Оценяващата функция може да е изцяло евристична (като при Greedy) или да е комбинация от евристична функция и такава, която дава точна оценка (като при A^*).

- **Greedy Best-first search**

Сортира списъка в съответствие с евристичната функция (например $h(\text{node}) = \text{<оценка на цената на пътя от node до целта>}$).

Оценка на метода:

- Непълен (опасност от зацикляне), неоптимален
- Времева сложност: $O(b^m)$, използването на подходяща евристика може да доведе до съществено подобрение
- Пространствена сложност: $O(b^m)$, съхраняват се всички достигнати състояния

- **Beam search**

Ограничава списъка до първите l най-добри възела от него (в съответствие с евристиката). При $l = 1$ се доближава до Hill climbing.

Оценка на метода:

- Непълен (опасност от зацикляне), неоптимален
- Времева сложност: $O(bl^m)$, зависи от разгледаното "изрязано" пространство и евристиката; използването на подходяща евристика може да доведе до съществено подобрение
- Пространствена сложност: $O(bl)$

- **Hill climbing**

Ограничава списъка до най-добрия му елемент (в съответствие с евристиката) и то само ако той е по-добър от своя родител. Търсенето е еднопосочно, без възможност за възврат.

Оценка на метода:

- Непълен (опасност от зацикляне), неоптимален
- Времева сложност: $O(bm)$, зависи от дълбочината на пространството и евристиката.
- Пространствена сложност: $O(b)$ - константа

- **A***

Комбиниращо търсене с равномерна цена на пътя с търсене на най-добър път. Списъкът се сортира в съответствие с функцията $f(\text{node}) = g(\text{node}) + h(\text{node})$. Функцията g връща като резултат цената на изминатия път от началния възел до node , а евристичната функция h връща като резултат приблизителна стойност на цената

на оставащата част от пътя от node до целта.

Оценка на метода:

- Пълен - ако разклоненията на всеки възел от графа на състоянията са краен брой и цената на преходите е положителна.
Оптимален - ако евристиката е приемлива (допустима), т.е. никога не надценява цената h^* на оставащия път (ако $h^*(node) \geq h(node)$ за всеки възел node).
- Времевата и пространствената сложност са приблизително $O(b^d)$, като зависят пряко от грешката на евристичната функция и дължината на най-късия път до решението.

Генетични алгоритми

Дефиниция **Генетичен алгоритъм** наричаме вариант на стохастично (локално) търсене в лъч, при което новите състояния се генерират чрез комбиниране на двойки родителски състояния.

Основни принципи:

- Състоянията се представят като низове над дадена крайна азбука (често като 0 и 1)
- Оценяващата функция (fitness function) оценява близостта до целта на дадено състояние, като има по-големи стойности за по-добрите състояния
- Алгоритъмът започва с множество (популация) от k случайно генерирани състояния (поколение 0)

Основни оператори:

- **Селекция** - оператор, чрез който се избират индивиди от текущото поколение за създаване на следващото. В генерирането на състоянията от следващото поколение участват някои от най-добрите представители на текущото поколение, избрани на случаен принцип.
- **Кръстосване** - оператор, чрез който се избират двойка "родителски" състояния и се определя т.н. точка на кръстосването им. Състоянието-наследник се получава чрез конкатенация на началната част на първия и крайната част на втория родител. Видове кръстосване:
 - В единична точка - избира се една точка на кръстосване и полученият низ от началото си до точката на кръстосване е копие на началната част на единия родител, а останалата му част е копие на съответната част на втория родител
 - В две точки - избират се две точки на кръстосване и полученият низ от началото си до първата точка на кръстосване е копие

на съответната част от първия родител, частта на резултата от първата точка на кръстосване до втората точка на кръстосване е копие на съответната част на втория родител и останалото е копие на оставащата след втората точка на кръстосване част на първия родител

– Аритметично - извършва се определена операция (аритметична, логическа, т.н.) между двамата родители

- **Мутация** - оператор, чрез който се извършват случайни промени в случайно избрана малка част от новата популация с цел да се осигури възможност за достигане на всяка точка от пространството на състоянията и да се избегне опасността от попадане в локален екстремум.